

Algoritmo de Dijkstra

El **algoritmo de Dijkstra** es un algoritmo que resuelve el problema del **camino más corto** en un grafo ponderado (con pesos en las aristas). Su objetivo es encontrar la ruta más corta desde un nodo origen a todos los demás nodos del grafo. Es ampliamente utilizado en sistemas de navegación, redes y otros problemas que requieren optimización de rutas.

Funcionamiento del Algoritmo

1. **Inicialización:** Se elige un nodo de inicio (origen). A todos los demás nodos se les asigna una distancia infinita, excepto al nodo de inicio, que tiene una distancia de 0.
2. **Exploración:** Se exploran los nodos adyacentes al nodo actual y se actualizan las distancias si se encuentra un camino más corto.
3. **Marcar como visitado:** Una vez que se ha encontrado el camino más corto a un nodo, ese nodo se marca como "visitado" y no se vuelve a procesar.
4. **Repetir:** El proceso continúa hasta que todos los nodos hayan sido visitados.

Restricciones

- El algoritmo de Dijkstra solo funciona con grafos cuyos pesos son **positivos**. Si hay pesos negativos, el algoritmo no garantiza encontrar el camino más corto correctamente.

Implementación en Python

A continuación, presento dos implementaciones del algoritmo de Dijkstra: una usando una **matriz de adyacencia** y otra usando una **lista de adyacencia**.

1. Implementación con Matriz de Adyacencia

```
import sys

# Función para encontrar el vértice con la distancia mínima
def min_distancia(distancias, visitados):
    min_val = sys.maxsize
    min_indice = -1
    for v in range(len(distancias)):
        if distancias[v] < min_val and not visitados[v]:
            min_val = distancias[v]
            min_indice = v
    return min_indice

# Implementación del algoritmo de Dijkstra usando matriz de adyacencia
def dijkstra_matriz(grafo, origen):
    n = len(grafo)
    distancias = [sys.maxsize] * n # Inicializamos las distancias a infinito
    distancias[origen] = 0 # La distancia al origen es 0
    visitados = [False] * n # Ningún nodo ha sido visitado aún
```

```

    for _ in range(n):
        u = min_distancia(distancias, visitados) # Encuentra el nodo con la
        distancia mínima
        visitados[u] = True # Marca el nodo como visitado

        # Actualiza las distancias de los nodos adyacentes no visitados
        for v in range(n):
            if grafo[u][v] > 0 and not visitados[v] and distancias[u] +
            grafo[u][v] < distancias[v]:
                distancias[v] = distancias[u] + grafo[u][v]

    return distancias

# Ejemplo
grafo_matriz = [
    [0, 10, 0, 30, 100],
    [10, 0, 50, 0, 0],
    [0, 50, 0, 20, 10],
    [30, 0, 20, 0, 60],
    [100, 0, 10, 60, 0]
]

origen = 0
distancias = dijkstra_matriz(grafo_matriz, origen)
print("Distancias mínimas desde el nodo", origen)
print(distancias)

```

Explicación

- Se utiliza una matriz para representar el grafo. Cada celda `grafo[i][j]` contiene el peso de la arista entre los nodos `i` y `j`.
- El algoritmo busca iterativamente el nodo con la menor distancia no visitada y actualiza las distancias para sus vecinos.

2. Implementación con Lista de Adyacencia

```

import heapq

# Implementación del algoritmo de Dijkstra usando lista de adyacencia
def dijkstra_lista(grafo, origen):
    distancias = {nodo: float('inf') for nodo in grafo} # Inicializamos las
    distancias a infinito
    distancias[origen] = 0
    cola_prioridad = [(0, origen)] # Cola de prioridad para seleccionar el
    nodo con menor distancia

    while cola_prioridad:
        distancia_actual, nodo_actual = heapq.heappop(cola_prioridad)

```

```

    if distancia_actual > distancias[nodo_actual]:
        continue

    # Explorar vecinos del nodo actual
    for vecino, peso in grafo[nodo_actual].items():
        distancia = distancia_actual + peso

        if distancia < distancias[vecino]:
            distancias[vecino] = distancia
            heapq.heappush(cola_prioridad, (distancia, vecino))

    return distancias

# Ejemplo
grafo_lista = {
    'A': {'B': 3, 'C': 2},
    'B': {'A': 3, 'C': 1, 'D': 5},
    'C': {'A': 2, 'B': 1, 'D': 4},
    'D': {'B': 5, 'C': 4}
}

origen = 'A'
distancias = dijkstra_lista(grafo_lista, origen)
print("Distancias mínimas desde el nodo", origen)
print(distancias)

```

Explicación

- El grafo está representado como un diccionario donde cada clave es un nodo y su valor es otro diccionario que contiene los vecinos y los pesos hacia esos vecinos.
- Se utiliza una cola de prioridad (implementada con `heapq`) para seleccionar eficientemente el siguiente nodo a explorar.

Comparación entre ambas implementaciones

Característica	Matriz de Adyacencia	Lista de Adyacencia
Estructura usada	Matriz $n \times n$	Diccionario (lista)
Complejidad Espacial	$O(n^2)$	$O(V + E)$
Complejidad Temporal	$O(n^2)$	$O((V + E) \log V)$
Uso recomendado	Grafos densos (muchas aristas)	Grafos dispersos (pocas aristas)
Eficiencia en grafos grandes	Menos eficiente debido al espacio cuadrático	Más eficiente debido a la cola de prioridad

Conclusión:

- La implementación con **lista de adyacencia** es generalmente más eficiente para grafos dispersos debido a su menor complejidad espacial y temporal.
- La implementación con **matriz de adyacencia** puede ser útil en grafos densos o cuando se necesita acceso directo entre cualquier par de nodos.